

MANUAL DEL PROGRAMADOR: APLICACIÓN DE LISTA DE TAREAS

1. Arquitectura y Componentes del Sistema

La aplicación está diseñada bajo una arquitectura monolítica frontend estructurada en tres capas tradicionales de la web (Separación de incumbencias o *Separation of Concerns*):

- **Estructura (HTML5):** Definida en `todo_app.html`. Utiliza etiquetas semánticas y atributos de accesibilidad (`aria-hidden`, `sr-only`).
- **Presentación (CSS3):** Vinculada a través de `/css/todo_app.css` para el diseño visual, diseño responsivo y estados de los componentes.
- **Lógica de Negocio y Control (JavaScript ES6+):** Orquestada de forma modular mediante un script de tipo módulo (`/todo_app.js`) que maneja el estado de la aplicación, la manipulación del DOM y la persistencia (generalmente *LocalStorage*).

2. Ciclo de Vida de la Aplicación

El ciclo de vida de la aplicación describe los estados y transiciones por los que pasa el sistema desde que el usuario interactúa con la URL hasta que se cierra o refresca la pestaña.

[Carga del DOM] → [Inicialización del Estado] → [Renderizado Inicial] → [Bucle de Eventos (Reactividad)]

Fase 1: Carga y Parseo del DOM

1. El navegador descarga y procesa `todo_app.html`.
2. Se procesan los enlaces de hojas de estilo (CSS).
3. Se detecta el tag `<script type="module" src="/todo_app.js">`. Al ser de tipo `module`, su ejecución se difiere (`deferred`) automáticamente hasta que el DOM esté completamente parseado.

Fase 2: Inicialización del Estado (Bootstrapping)

Al ejecutarse `todo_app.js`, se deben realizar las siguientes acciones de configuración inicial:

- **Carga de Datos:** Recuperar el array de tareas existente desde la persistencia (ej. `localStorage.getItem('tasks')`). Si no existe, inicializar un array vacío.
- **Mapeo de UI:** Guardar referencias a los nodos del DOM clave (`#task-input`, `#tasks-list`, los contadores de estadísticas, etc.).
- **Fecha Actual:** Inyectar la fecha del día en el elemento `#date-label`.
- **Registro de Escuchadores (Event Listeners):** Configurar los capturadores de clicks y teclas (`click` en el botón añadir, `keypress "Enter"` en el input, clicks en la barra de filtros).

Fase 3: Renderizado Inicial

- Se ejecuta la función principal de renderizado (`render()`), la cual lee el estado de las tareas actual, aplica el filtro por defecto (`all`) y genera dinámicamente los elementos HTML dentro del contenedor `#tasks-list`.
- Se recalculan las estadísticas iniciales (Total, Pendientes, Completadas).

Fase 4: Bucle de Eventos y Mutación del Estado (Reactividad)

La aplicación entra en un estado de espera activa. Cada acción del usuario (crear, borrar, marcar como completada, filtrar) sigue el flujo unidireccional:

- El usuario interactúa con la UI → 2. Se dispara el evento JavaScript → 3. Se modifica el array de datos (Estado) y se guarda en almacenamiento → 4. Se vuelve a invocar a la función de renderizado para actualizar la UI.

3. Matriz de Casos de Uso

A continuación se detallan los casos de uso técnicos basados en los elementos definidos en el archivo HTML.

ID	Caso de Uso	Actores / Disparadores	Elementos de UI Involucrados	Precondición	Flujo Básico (Lógica Técnica)
CU-01	Crear Tarea	Usuario / Click o Enter	#task-input, #cat-select, #prio-select, #add-btn	El campo de texto no debe estar vacío.	1. Capturar valores de los inputs. 2. Crear objeto JSON con un ID único (crypto.randomUUID() o Date.now()), texto, categoría, prioridad y estado done: false. 3. Hacer .push() al estado. 4. Guardar y renderizar.
CU-02	Visualizar Estadísticas	Sistema / Automático	#s-total, #s-pending, #s-done	Cambio en el estado de las tareas.	1. Recorrer el array de tareas. 2. Filtrar el total, las que tienen done === false y las que tienen done === true. 3. Actualizar el .textContent de cada nodo correspondiente.

ID	Caso de Uso	Actores / Disparadores	Elementos de UI Involucrados	Precondición	Flujo Básico (Lógica Técnica)
CU-03	Filtrar Tareas	Usuario / Click	.filter-btn, #tasks-list	Existencia de tareas en el estado.	1. Capturar el atributo <code>data-filter</code> del botón clickeado. 2. Remover clase <code>.active</code> de los otros botones y añadirla al seleccionado. 3. Filtrar el array de tareas según el criterio (por estado o por categoría). 4. Renderizar solo el subconjunto.
CU-04	Conmutar Estado (Check/Uncheck)	Usuario / Click	Elemento dinámico dentro de #tasks-list	La tarea debe existir en la interfaz.	1. Capturar el ID de la tarea asociada mediante un atributo <code>data-id</code>. 2. Invertir el valor booleano de la propiedad <code>done</code>. 3. Guardar, recalcular estadísticas y re-renderizar.
CU	Eliminar	Usuario	Botón de	La tarea	1. Capturar el ID

ID	Caso de Uso	Actores / Dispara dores	Elementos de UI Involucra dos	Precondi ción	Flujo Básico (Lógica Técnica)
-05	Tarea	/ Click	borrado dinámico en #tasks- list	debe existir en la interfaz.	de la tarea a través de data-id. 2. Filtrar el array excluyendo dicho ID (tasks.filter()). 3. Guardar en almacenamiento y re-renderizar la lista.